

Node服务器 - koa框架

王红元 coderwhy

目录

content



1 koa的基本使用

2 koa的参数解析

3 koa响应和错误

4 koa静态服务器

5 koa的源码解析

6 和express对比

■ 前面我们已经学习了express，另外一个非常流行的Node Web服务器框架就是Koa。

■ Koa官方的介绍：

- koa: next generation web framework for node.js;

- koa: node.js的下一代web框架;

■ 事实上，koa是express同一个团队开发的一个新的Web框架：

- 目前团队的核心开发者TJ的主要精力也在维护Koa，express已经交给团队维护了；

- Koa旨在为Web应用程序和API提供更小、更丰富和更强大的能力；

- 相对于express具有更强的异步处理能力（后续我们再对比）；

- Koa的核心代码只有1600+行，是一个更加轻量级的框架；

- 我们可以根据需要使用中间件；

■ 事实上学习了express之后，学习koa的过程是很简单的；

Koa初体验

■ 我们来体验一下koa的Web服务器，创建一个接口。

□ koa也是通过注册中间件来完成请求操作的；

■ koa注册的中间件提供了两个参数：

■ ctx：上下文（Context）对象；

□ koa并没有像express一样，将req和res分开，而是将它们作为ctx的属性；

□ ctx代表一次请求的上下文对象；

□ ctx.request：获取请求对象；

□ ctx.response：获取响应对象；

■ next：本质上是一个dispatch，类似于之前的next；

□ 后续我们学习Koa的源码，来看一下它是一个怎样的函数；

```
const Koa = require('koa');

const app = new Koa();

app.use((ctx, next) => {
  console.log("middleware 01");
  next();
})

app.use((ctx, next) => {
  console.log("middleware 02");
  ctx.response.body = "Hello World";
})

app.listen(8000, () => {
  console.log("服务器启动成功~");
});
```

Koa中间件

■ koa通过创建的app对象，注册中间件只能通过use方法：

- Koa并没有提供methods的方式来注册中间件；
- 也没有提供path中间件来匹配路径；

■ 但是真实开发中我们如何将路径和method分离呢？

- 方式一：根据request自己来判断；
- 方式二：使用第三方路由中间件；

```
app.use((ctx, next) => {  
  if (ctx.request.path === '/users') {  
    if (ctx.request.method === 'POST') {  
      ctx.response.body = "Create User Success~";  
    } else {  
      ctx.response.body = "Users List~";  
    }  
  } else {  
    ctx.response.body = "Other Request Response";  
  }  
})
```

路由的使用

- koa官方并没有给我们提供路由的库，我们可以选择第三方库：

koa-router

```
npm install @koa/router
```

- 我们可以先封装一个 user.router.js 的文件：
- 在app中将router.routes()注册为中间件：
- 注意：allowedMethods用于判断某一个method是否支持：
 - 如果我们请求 get，那么是正常的请求，因为我们有实现 get；
 - 如果我们请求 put、delete、patch，那么就自动报错：Method Not Allowed，状态码：405；
 - 如果我们请求 link、copy、lock，那么久自动报错：Not Implemented，状态码：501；

```
const userRouter = new KoaRouter({ prefix: '/users' })

userRouter.get('/', (ctx, next) => {
  ctx.body = '用户列表~'
})

userRouter.post('/', (ctx, next) => {
  ctx.status = 201
  ctx.body = '用户创建~'
})

app.use(userRouter.routes())
```

```
app.use(userRouter.routes());
app.use(userRouter.allowedMethods());
```

参数解析: params - query

■ 请求地址: <http://localhost:8000/users/123>

□ 获取params:

```
const userRouter = new Router({prefix: "/users"})

userRouter.get("/:id", (ctx, next) => {
  console.log(ctx.params.id);
  ctx.body = "Hello World";
})
```

■ 请求地址: <http://localhost:8000/login?username=why&password=123>

□ 获取query:

```
app.use((ctx, next) => {
  console.log(ctx.request.query);
  ctx.body = "Hello World";
})
```

参数解析：json

■ 请求地址：<http://localhost:8000/login>

■ body是json格式：

```
{  
  "username": "coderwhy",  
  "password": "123"  
}
```

■ 获取json数据：

□ 安装依赖： `npm install koa-bodyparser;`

□ 使用 koa-bodyparser的中间件；

```
app.use(bodyParser());  
  
app.use((ctx, next) => {  
  console.log(ctx.request.body);  
  ctx.body = "Hello World";  
})
```


参数解析: x-www-form-urlencoded

■ 请求地址: <http://localhost:8000/login>

□ body是x-www-form-urlencoded格式:

The screenshot shows a REST client interface with a POST request to `http://localhost:8000/login`. The 'Body' tab is selected, and the format is set to 'x-www-form-urlencoded'. A table lists the request body parameters:

	KEY	VALUE	DESCRIPTION
☒	username	kobe	
☒	password	8888	

Below the table, there are labels 'Key', 'Value', and 'Description'.

■ 获取json数据: (和json是一致的)

□ 安装依赖: `npm install koa-bodyparser;`

□ 使用 koa-bodyparser的中间件;

```
app.use((ctx, next) => {  
  console.log(ctx.request.body);  
  ctx.body = "Hello World";  
})
```

参数解析: form-data

■ 请求地址: <http://localhost:8000/login>

□ body是form-data格式

The screenshot shows a REST client interface for a POST request to `http://localhost:8000/login`. The 'Body' tab is selected, and the 'form-data' radio button is chosen. Below, a table lists the form fields:

	KEY	VALUE	DESCRIPTION
☒	username	lilei	
☒	password	8888	
	Key	Value	Description

■ 解析body中的数据, 我们需要使用multer

□ 安装依赖: `npm install koa-multer;`

□ 使用 multer中间件;

```
const upload = multer({
});

app.use(upload.any());

app.use((ctx, next) => {
  console.log(ctx.req.body);
  ctx.body = "Hello World";
});
```

Multer上传文件

```
const storage = multer.diskStorage({
  destination: (req, file, cb) => {
    cb(null, './uploads/')
  },
  filename: (req, file, cb) => {
    cb(null, Date.now() + path.extname(file.originalname))
  }
})

const upload = multer({
  storage
});

const fileRouter = new Router();

fileRouter.post("/upload", upload.single('avatar'), (ctx, next) => {
  console.log(ctx.req.file);
})
```

- koa并没有内置部署相关的功能，所以我们需要使用第三方库：

```
npm install koa-static
```

- 部署的过程类似于express：

```
const Koa = require('koa');
const static = require('koa-static');

const app = new Koa();

app.use(static('./build'));

app.listen(8000, () => {
  console.log("静态服务器启动成功~");
});
```

■ 输出结果：body将响应主体设置为以下之一：

- string：字符串数据
- Buffer：Buffer数据
- Stream：流数据
- Object|| Array：对象或者数组
- null：不输出任何内容
- 如果response.status尚未设置，Koa会自动将状态设置为200或204。

■ 请求状态：status

```
ctx.response.body = "Hello World";  
ctx.body = {  
  name: "why",  
  age: 18,  
  height: 1.88  
};  
ctx.body = ["abc", "cba", "nba"];
```

```
ctx.status = 201;  
ctx.response.status = 204;
```

错误处理

```
const Koa = require('koa');

const app = new Koa();

app.use((ctx, next) => {
  ctx.app.emit('error', new Error("哈哈"), ctx);
})

app.on('error', (err, ctx) => {
  console.log(err.message);
  ctx.response.body = "哈哈";
})

app.listen(8000, () => {
  console.log("错误处理服务启动成功~");
})
```

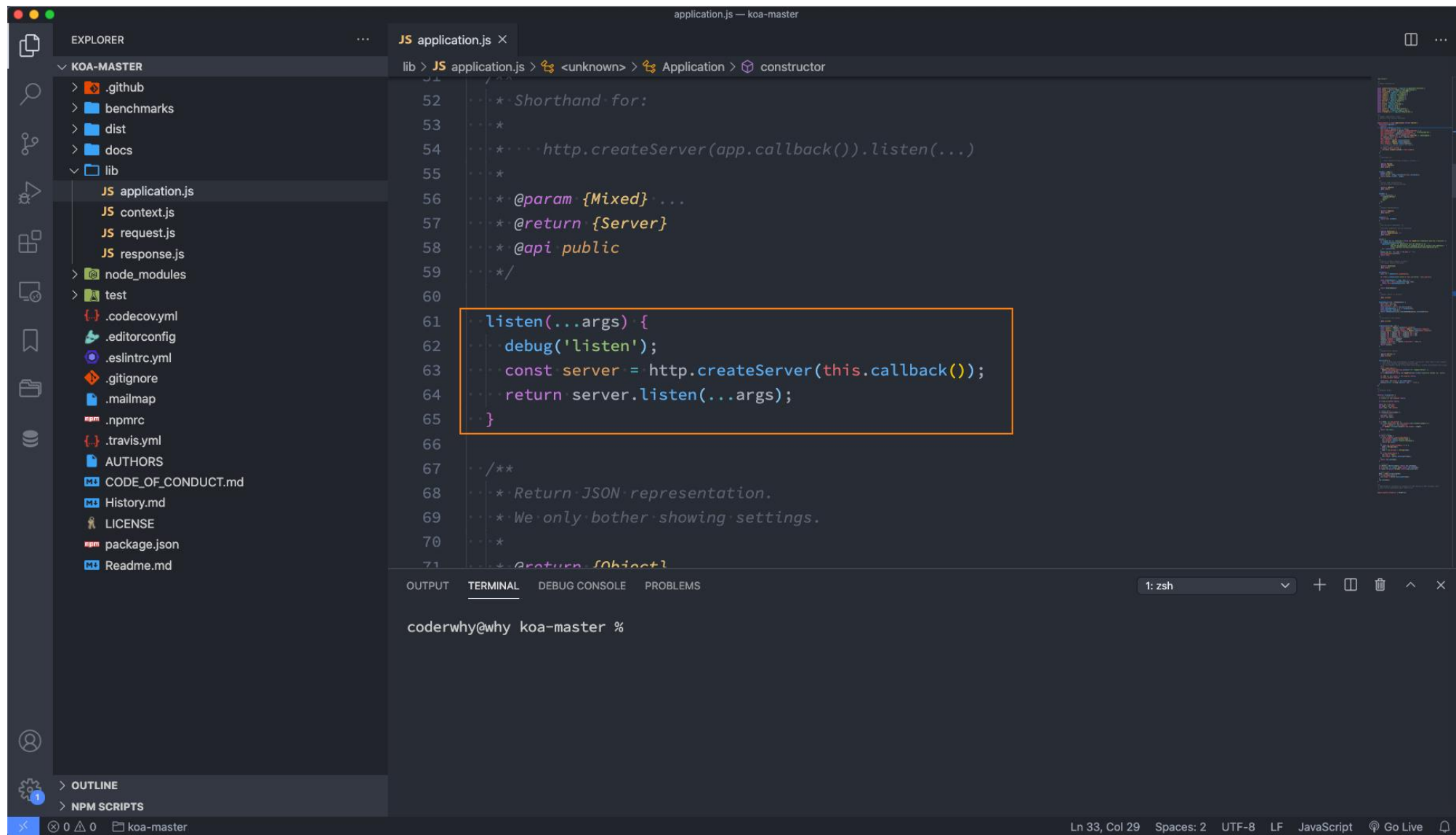
创建Koa的过程

The screenshot shows a VS Code editor window with the file explorer on the left, the editor in the center, and a terminal at the bottom. The file explorer shows a project named 'koa-master' with a 'lib' directory containing 'application.js', 'context.js', 'request.js', and 'response.js'. The editor is open to 'application.js', showing the following code:

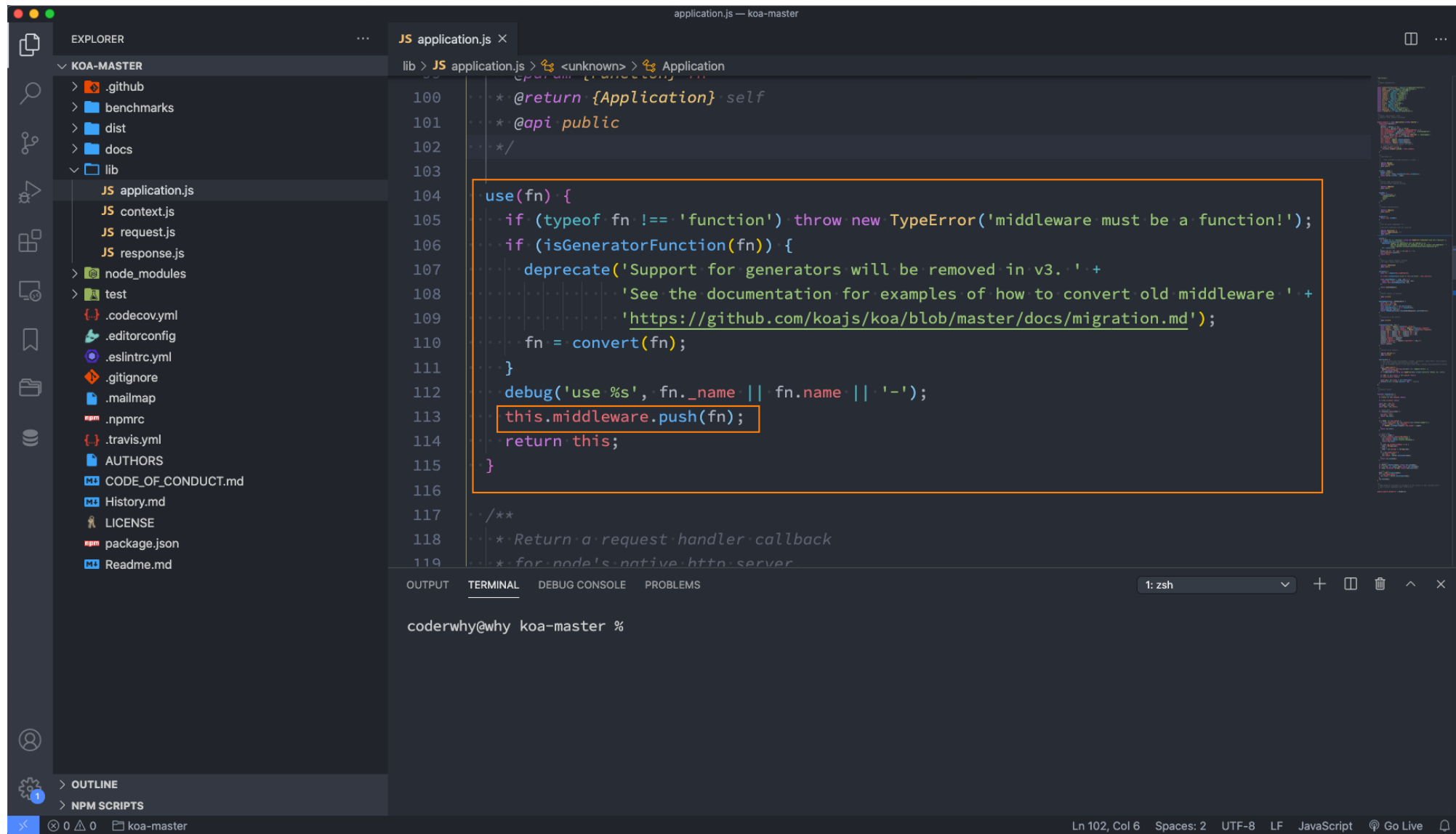
```
lib > JS application.js > <unknown> > Application > constructor
20 // Expose Application class.
21
22 /**
23  * Inherit from `Emitter.prototype`..
24  */
25
26
27
28
29
30 module.exports = class Application extends Emitter {
31   constructor(options) {
32     super();
33     options = options || {};
34     this.proxy = options.proxy || false;
35     this.subdomainOffset = options.subdomainOffset || 2;
36     this.proxyIpHeader = options.proxyIpHeader || 'X-Forwarded-For';
37     this.maxIpsCount = options.maxIpsCount || 0;
38     this.env = options.env || process.env.NODE_ENV || 'development';
39     if (options.keys) this.keys = options.keys;
40     this.middleware = [];
41     this.context = Object.create(context);
42     this.request = Object.create(request);
43     this.response = Object.create(response);
44     // util.inspect.custom support for node 6+
45     /** istanbul ignore else */
46     if (util.inspect.custom) {
```

The terminal at the bottom shows the command prompt: `coderwhy@why koa-master %`. The status bar at the bottom indicates the current position is Line 32, Column 13, with 2 spaces, UTF-8 encoding, LF line endings, and the file is a JavaScript file.

开启监听



注册中间件



监听回调

The screenshot shows the VS Code editor interface with the file `application.js` open. The Explorer sidebar on the left shows the project structure for `KOA-MASTER`, including files like `context.js`, `request.js`, `response.js`, and `node_modules`. The main editor area displays the `application.js` file with the following code:

```
120  *
121  * @return {Function}
122  * @api public
123  */
124
125  callback() {
126    const fn = compose(this.middleware);
127
128    if (!this.listenerCount('error')) this.on('error', this.onerror);
129
130    const handleRequest = (req, res) => {
131      const ctx = this.createContext(req, res);
132      return this.handleRequest(ctx, fn);
133    };
134
135    return handleRequest;
136  }
137
138  /**
139   * Handle request in callback.
```

Annotations in the code include:

- A yellow box around `this.callback()` in line 126.
- A yellow box around `return handleRequest;` in line 135.
- An orange arrow pointing from the `return handleRequest;` line to the `const ctx = this.createContext(req, res);` line in the `handleRequest` function.

The bottom of the editor shows the TERMINAL panel with the prompt `coderwhy@why koa-master %`. The status bar at the bottom indicates the current position is `Ln 125, Col 3` in `JavaScript` file.

compose方法

The screenshot shows a VS Code editor window with the file explorer on the left and the code editor in the center. The file explorer shows a project named 'koa-master' with various folders and files. The code editor displays the 'index.js' file, which contains the implementation of the 'compose' method. The code is as follows:

```
14  * @param {Array} middleware
15  * @return {Function}
16  * @api public
17  */
18
19  function compose (middleware) {
20    /** 判断middleware如果不是数组, 抛出异常
21     * if (!Array.isArray(middleware)) throw new TypeError('Middleware stack must be an array!')
22     * // middleware的原生如果不是函数, 抛出异常
23     * for (const fn of middleware) {
24       if (typeof fn !== 'function') throw new TypeError('Middleware must be composed of functions!')
25     }
26
27     /**
28     * @param {Object} context
29     * @return {Promise}
30     * @api public
31     */
32
33     return function (context, next) {
34       /** last called middleware #
35       * let index = -1
36       * return dispatch(0)
37       * function dispatch (i) {
38         if (i <= index) return Promise.reject(new Error('next() called multiple times'))
39         index = i
40         let fn = middleware[i]
```

和express对比

- 在学习了两个框架之后，我们应该已经可以发现koa和express的区别：
- 从架构设计上来说：
 - express是完整和强大的，其中帮助我们内置了非常多好用的功能；
 - koa是简洁和自由的，它只包含最核心的功能，并不会对我们使用其他中间件进行任何的限制。
 - 甚至是在app中连最基本的get、post都没有给我们提供；
 - 我们需要通过自己或者路由来判断请求方式或者其他功能；
- 因为express和koa框架他们的核心其实都是中间件：
 - 但是他们的中间件事实上，它们的中间件的执行机制是不同的，特别是针对某个中间件中包含异步操作时；
 - 所以，接下来，我们再来研究一下express和koa中间件的执行顺序问题；

■ 我通过一个需求来演示所有的过程：

- 假如有三个中间件会在一次请求中匹配到，并且按照顺序执行；
- 我希望最终实现的方案是：
 - ✓ 在middleware1中，在req.message中添加一个字符串 aaa；
 - ✓ 在middleware2中，在req.message中添加一个 字符串bbb；
 - ✓ 在middleware3中，在req.message中添加一个 字符串ccc；
 - ✓ 当所有内容添加结束后，在middleware1中，通过res返回最终的结果；

■ 实现方案：

- Express同步数据的实现；
- Express异步数据的实现；
- Koa同步数据的实现；
- Koa异步数据的实现；

■ 具体的代码查看课堂演练。

koa洋葱模型

- 两层理解含义:
- 中间件处理代码的过程;
- Response返回body执行;

